

Bare-Metal Type-1 Hypervisor for Autonomous Vehicle Platform

ARMv8-A

Problem Statement

Autonomous vehicles require executing safety-critical applications (such as steering control loops) alongside non-critical background applications (such as telemetry loggers or OTA update services). Running both within a single OS kernel risks catastrophic failure if the non-critical application encounters a memory leak, kernel panic, or malicious exploit. RuncHypervisor solves this by implementing a Type-1 bare-metal hypervisor directly at Exception Level 2 (EL2) on ARMv8-A. It enforces strict hardware-level memory isolation via Stage-2 page tables and utilizes a mixed-criticality scheduler to guarantee deterministic execution windows for the safety-critical OS, ensuring that secondary systems can never starve the autonomous driving capabilities.

System Overview

Key	Value
Target Hardware	ARMv8-A Cortex-A57 QEMU virt platform
Privilege Level	Exception Level 2 (EL2) for Hypervisor, EL1 for Guests
Timer & Tick Rate	62.5 MHz physical generic timer (CNTHP_TVAL_EL2)
Memory Management	3-level Stage-2 MMU translation
Guest Physical Space	112 MB isolated execution RAM (0x48000000 to 0x4F000000)

Context Switch State	34 architectural registers saved (304 bytes total overhead)
Device Emulation	Trap-and-emulate MMIO over Virtual PL011 UART
Scheduling Model	Deterministic asymmetric quanta (64ms RTOS / 16ms Logger)
Stdlib / Runtime	None — compiled freestanding with custom assembly exception vectors

Workflow Overview

Execution fundamentally resides within the Guest OS (EL1) until a hardware trap forces control back to the Hypervisor (EL2). This trap can be an asynchronous physical timer interrupt triggering a context switch, or a synchronous Data Abort caused by the guest attempting to write to a protected memory region.

When a trap occurs, the hardware vector table immediately branches to a custom assembly macro that saves the complete state of the faulting VM. The C-based hypervisor logic then categorizes the Exception Syndrome Register (ESR_EL2) to determine the exact cause of the exit.

The hypervisor execution pipeline processes strict priority paths: CONTEXT SAVE — 304 bytes pushed to EL2 stack in <1 us EXCEPTION DECODE — Parses Exception Class (EC) from ESR_EL2 SCHEDULER TICK (if IRQ) — Evaluates 16ms/64ms quanta and swaps the active vCPU state MMIO EMULATION (if Sync) — Decodes Data Abort addressing and routes characters to hardware CONTEXT RESTORE — Pops 34 registers and executes `eret` to resume the guest

Memory Layout The memory architecture separates the physical address space into distinct zones. The Stage-2 MMU acts as an invisible hardware barrier; the guest OS operates under the illusion that it controls physical memory, while the hypervisor actively translates the guest's Intermediate Physical Addresses (IPA) to true physical addresses.

A 1 MB "Forbidden Zone" is deliberately left unmapped to immediately trap anomalous execution behavior. Memory mapping operations are finalized with a Point-of-Coherency cache flush (dc civac) to ensure the hardware page table walker reads accurate data directly from RAM.

Physical Memory Map (40-bit IPA): 0x09000000 : VIRTUAL UART (4 KB) — Unmapped Stage-2, triggers MMIO trap
0x40000000 : HYPERVISOR BASE — L1/L2/L3 Page Tables & EL2 code
0x48000000 : GUEST MEMORY BASE (112 MB) — Mapped RW+X
0x4FF00000 : FORBIDDEN ZONE (1 MB) — Unmapped memory boundary

Boot Sequence The boot sequence establishes the secure execution environment before any guest code is loaded. Upon execution, the hypervisor verifies it is running at EL2 and initializes the UART for debugging. It then installs the 16-entry AArch64 exception vector table by modifying the vbar_el2 register.

Before activating the hardware MMU, guest machine-code stubs are loaded directly into the targeted guest RAM sectors. The VTCR_EL2 register is configured for a 40-bit Intermediate Physical Address space, and the VTTBR_EL2 register is pointed to the zeroed L1 translation table. The hypervisor enables Stage-2 translation by asserting the VM bit in HCR_EL2, permanently sandboxing all future EL1 execution.

Virtual Device Emulation (MMIO) Because peripheral hardware like the UART serial port cannot be safely shared between multiple concurrent operating systems, the hypervisor utilizes Memory-Mapped I/O (MMIO) emulation. The physical memory region for the PL011 UART is intentionally excluded from the guest's Stage-2 page tables.

When the guest attempts to print a character, the CPU throws a Stage-2 Data Abort. The hypervisor catches this exception and parses the ESR_EL2 register to extract the Instruction Syndrome Valid (ISV) flag, the Write/Not-Read (WnR) flag, and the Syndrome Register Transfer (SRT). By reading the specific general-purpose register the guest used for the faulting instruction, the hypervisor extracts the intended character, writes it safely to the actual hardware UART, and advances the guest's program counter (elr_el2) by 4 bytes to seamlessly resume execution.

```
// Exception Syndrome Register (ESR_EL2) Bit Extraction
#define ESR_ISV(esr) (((esr) >> 24) & 0x1)
#define ESR_WNR(esr) (((esr) >> 6) & 0x1)
#define ESR_SRT(esr) (((esr) >> 16) & 0x1F)
```

```
// Extracting guest payload during a trapped UART write
unsigned int srt = ESR_SRT(esr);
unsigned long val = ctx_read_reg(ctx, srt);
char ch = (char)(val & 0xFF);
uart_putc(ch);
ctx->elr_el2 += 4; // Advance past faulting instruction
```

Context Switching & Exceptions The context switch engine is written in pure AArch64 assembly to guarantee deterministic, sub-microsecond latency. The

SAVE_GUEST_CONTEXT macro decrements the stack pointer by 304 bytes to allocate space for 31 general-purpose registers (X0-X30), the stack pointers (sp_el0, sp_el1), and the critical hardware exception registers (elr_el2, spsr_el2, esr_el2, far_el2).

Because the exception vectors are strictly aligned to 0x80 byte boundaries in memory, the hardware jumps directly to the correct handler based on whether the exception originated from the current Exception Level or a lower one.

```
.macro SAVE_GUEST_CONTEXT sub sp, sp, #CTX_SIZE stp x0, x1, [sp, #0] // ... saves  
all 31 registers ... mrs x0, elr_el2 str x0, [sp, #264] mrs x0, esr_el2 str x0, [sp, #280]  
.endm
```

Sample Output

RuncHypervisor Mixed-Criticality Scheduling VM0: RuncOS | VM1: Logger

[BOOT] Exception Level: EL2

[BOOT] Exception vectors installed

[MMU] Writing guest stubs (Stage-2 inactive)...

[MMU] Configuring Stage-2 translation...

[MMU] VTTBR_EL2 loaded, L1 table at: 0x40020000

[MMU] Mapping guest memory RW+X: 0x48000000 – 0x4F000000

[MMU] Virtual UART region unmapped (MMIO trap): 0x09000000

[MMU] Stage-2 translation ENABLED (HCR_EL2.VM=1)

[TEST] Checking forbidden page entry...

[MMU] Lookup IPA: 0x4FF00000

[MMU] L1: NOT MAPPED

[TEST] Forbidden page confirmed NOT MAPPED.

Launching Mixed-Criticality Scheduler

[MMIO] Virtual UART fault

[MMIO] FAR offset: 0x000

[MMIO] Guest wrote char: 'S' via X0

[MMIO] Guest wrote char: 'Y' via X0

[MMIO] Guest wrote char: 'S' via X0

Build & Run

Compile (AArch64 bare-metal cross-compilation)

```
aarch64-elf-gcc -mcpu=cortex-a57 -ffreestanding  
-nostdlib -nostartfiles -T linker.ld
```

```
exceptions.S main.c mmu.c mmio.c uart.c -o hypervisor.elf
```

Run in QEMU virt platform (virtualization enabled)

```
qemu-system-aarch64 -machine virt,virtualization=on -cpu cortex-a57  
-nographic -kernel hypervisor.elf
```

Conclusion

RuncHypervisor proves the viability of constructing a highly secure, deterministic hardware execution environment entirely from scratch on complex modern architectures. By leveraging AArch64 hardware virtualization extensions, Stage-2 page tables, and cycle-accurate generic timers, the hypervisor cleanly abstracts the underlying SoC. This allows safety-critical control loops and non-critical data infrastructure to securely share a single CPU core with zero risk of cross-domain memory corruption or execution starvation, forming a robust foundation for modern autonomous vehicle platforms.